

@app — pipeline

Appears in: [DL-01](#)

app.py:89-130

footnote ¹ — "The agency-function reasoning is set out in the gap analysis; the running pipeline that realises it is @app." (body: "The pipeline runs end to end on a product image with no API errors and no timeout, in under 120 seconds.")

@app — pipeline

app.py:89-130

```
89     def _stream():
90         t_start = time.time()
91         try:
92             yield 'data: {"step":"vision","label":"Analyzing product image..."}\n\n'
93             color_data = color.extract(image_path)
94             vision_data = vision.analyze(image_path, description)
95
96             yield 'data: {"step":"audience","label":"Segmenting target audience..."}\n\n'
97             rag_query = description + " " + " ".join(vision_data.get("product_tags", []))
98             doc_context = rag.retrieve(rag_query)
99             audience_data = audience.segment(description, vision_data, doc_context=doc_context)
100
101             yield 'data: {"step":"copywriter","label":"Writing ad variants..."}\n\n'
102             raw_copy = copywriter.generate(
103                 description, vision_data, color_data, tone, audience_data, doc_context=doc_context
104             )
105
106             yield 'data: {"step":"guardrails","label":"Checking compliance..."}\n\n'
107             result = guardrails.check(raw_copy)
108
109             yield 'data: {"step":"cta","label":"Optimizing CTAs..."}\n\n'
110             cta_result = cta_optimizer.optimize(
111                 result["clean_copy"], tone, audience_data, doc_context=doc_context
112             )
113
114             pipeline_s = time.time() - t_start
115             output = {
116                 "run_id": run_id,
117                 "meta": {"tone": tone, "description": description},
118                 "vision": vision_data,
119                 "colors": color_data,
120                 "audience": audience_data,
121                 "ads": result["clean_copy"],
122                 "cta_analysis": cta_result,
123                 "compliance_flags": result["flags"],
124                 "rag_chunks_used": len(doc_context),
125                 "pipeline_s": round(pipeline_s, 1),
126             }
127             _run_store[run_id] = {"output": output, "image_filename": filename}
128             _log_run(output, pipeline_s)
129
130             yield f'data: {{{{"step":"done","run_id":"{run_id}","pipeline_s":{round(pipeline_s,1)}}}}\n\n'
```

What it shows: The Flask /generate handler runs vision → audience → copywriter → guardrails → cta_optimizer in sequence — the running proof that the five-agent pipeline executes end to end.

Source: app.py:89-130 · image rendered by build_snippets.py